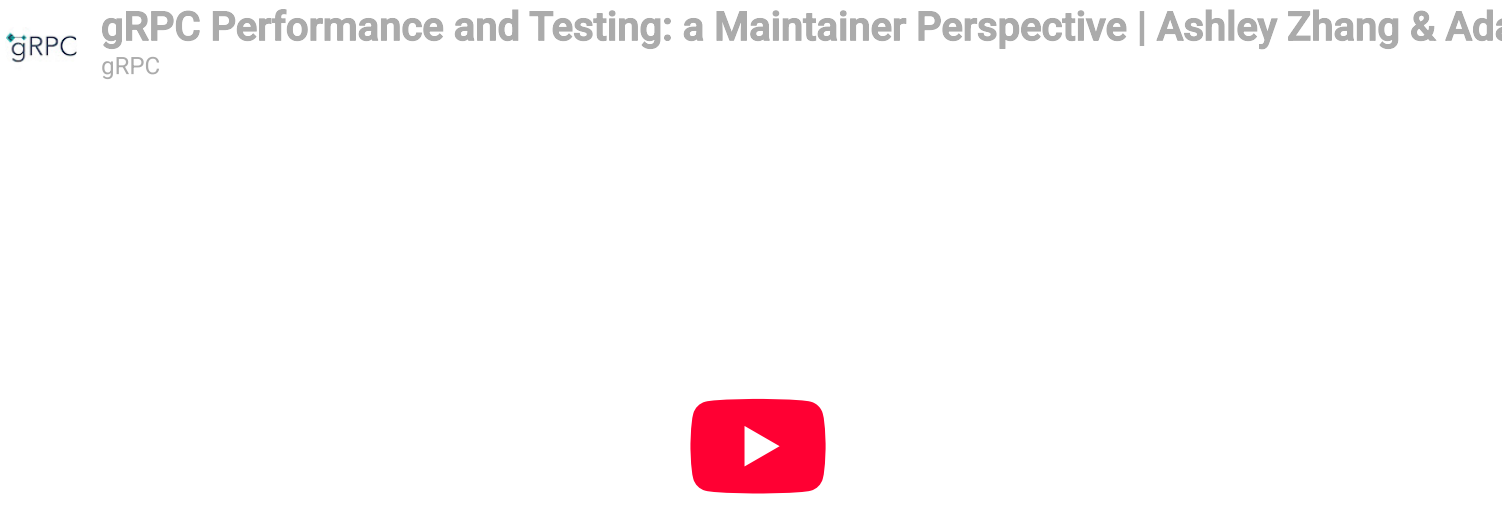
gRPConf India 2026 is on **Nov 18th at Bengaluru!** - **Register now** . Checkout **Schedule**

Performance Best Practices

A user guide of both general and language-specific best practices to improve performance.



General

- Always **re-use stubs and channels** when possible.
- **Use keepalive pings** to keep HTTP/2 connections alive during periods of inactivity to allow initial RPCs to be made quickly without a delay (i.e. C++ channel arg GRPC_ARG_KEEPALIVE_TIME_MS).
- **Use streaming RPCs** when handling a long-lived logical flow of data from the client-to-server, server-to-client, or in both directions. Streams can avoid continuous RPC initiation, which includes connection load balancing at the client-side, starting a new HTTP/2 request at the transport layer, and invoking a user-defined method handler on the server side.

  Streams, however, cannot be load balanced once they have started and can be hard to debug for stream failures. They also might increase performance at a small scale but can reduce scalability due to load balancing and complexity, so they should only be used when they provide substantial performance or simplicity benefit to application logic. Use streams to optimize the application, not gRPC.

  Side note: *This does not apply to Python (see Python section for details).*

- *(Special topic)* Each gRPC channel uses 0 or more HTTP/2 connections and each connection usually has a limit on the number of concurrent streams. When the number of active RPCs on the connection reaches this limit, additional RPCs are queued in the client and must wait for active RPCs to finish before they are sent. Applications with high load or long-lived streaming RPCs might see performance issues because of this queueing. There are two possible solutions:

  1. **Create a separate channel for each area of high load** in the application.
  2. **Use a pool of gRPC channels** to distribute RPCs over multiple connections (channels must have different channel args to prevent re-use so define a use-specific channel arg such as channel number).

  Side note: *The gRPC team has plans to add a feature to fix these performance issues (see grpc/grpc#21386 for more info), so any solution involving creating multiple channels is a temporary workaround that should eventually not be needed.*

C++

- **Do not use Sync API for performance sensitive servers.** If performance and/or resource consumption are not concerns, use the Sync API as it is the simplest to implement for low-QPS services.

- **Favor callback API over other APIs for most RPCs**, given that the application can avoid all blocking operations or blocking operations can be moved to a separate thread. The callback API is easier to use than the completion-queue async API but is currently slower for truly high-QPS workloads.
- If having to use the async completion-queue API, the **best scalability trade-off is having `numcpu` 's threads.** The ideal number of completion queues in relation to the number of threads can change over time (as gRPC C++ evolves), but as of gRPC 1.41 (Sept 2021), using 2 threads per completion queue seems to give the best performance.
- For the async completion-queue API, make sure to **register enough server requests for the desired level of concurrency** to avoid the server continuously getting stuck in a slow path that results in essentially serial request processing.
- *(Special topic)* gRPC::GenericStub can be useful in certain cases when there is high contention / CPU time spent on proto serialization. This class allows the application to directly send **raw gRPC::ByteBuffer as data** rather than serializing from some proto. This can also be helpful if the same data is being sent multiple times, with one explicit proto-to-ByteBuffer serialization followed by multiple ByteBuffer sends.

Java

- **Use non-blocking stubs** to parallelize RPCs.
- **Provide a custom executor that limits the number of threads, based on your workload** (cached (default), fixed, forkjoin, etc).

Python

- Streaming RPCs create extra threads for receiving and possibly sending the messages, which makes **streaming RPCs much slower than unary RPCs** in gRPC Python, unlike the other languages supported by gRPC.
- **Using asyncio** could improve performance.
- Using the future API in the sync stack results in the creation of an extra thread. **Avoid the future API** if possible.
- *(Experimental)* An experimental **single-threaded unary-stream implementation** is available via the SingleThreadedUnaryStream channel option, which can save up to 7% latency per message.

Last modified November 12, 2024: Embed YouTube videos in different webpages (#1380) (196f408)

View page source Edit this page Create child page Create documentation issue Create project issue